

Dense Associative Memories and Deep Feedforward Neural Networks Implemented Via Quantum Annealer

Master's Degree in Artificial Intelligence Systems
Academic year 2023/2024

Supervisor
Davide Pastorello

Student
Dimitri Vinci

Co-Supervisor
Enrico Blanzieri
Enrico Zardini



Why dense associative networks in the annealer



No quantum memory capable of exponential storage on annealer

-
1. C. Bauckhage, R. Sanchez, and R. Sifa. Problem solving with hopfield networks and adiabatic quantum computing. 2020.
 2. M. Schuld, I. Sinayskiy, and F. Petruccione. The quest for a quantum neural network. Quantum Information Processing, 2014
 3. C.F. Higham and A. Bedford. Quantum deep learning: Sampling neural nets with a quantum annealer. Nature Scientific Reports, 2021.
 4. M. Sasdelli and T. Chin. Quantum annealing formulation for binary neural networks. Proceedings of the Digital Image Computing: Techniques and Applications, 2021.

Why dense associative networks in the annealer

- ⇒ No quantum memory capable of exponential storage on annealer
- ⇒ Interest in neuroscientific community for associative memories due to biological plausibility
 - ⇒ Easy understandable out of computer scientist community ¹

1. C. Bauckhage, R. Sanchez, and R. Sifa. Problem solving with hopfield networks and adiabatic quantum computing. 2020.

2. M. Schuld, I. Sinayskiy, and F. Petruccione. The quest for a quantum neural network. Quantum Information Processing, 2014

3. C.F. Higham and A. Bedford. Quantum deep learning: Sampling neural nets with a quantum annealer. Nature Scientific Reports, 2021.

4. M. Sasdelli and T. Chin. Quantum annealing formulation for binary neural networks. Proceedings of the Digital Image Computing: Techniques and Applications, 2021.

Why dense associative networks in the annealer

- ⇒ No quantum memory capable of exponential storage on annealer
- ⇒ Interest in neuroscientific community for associative memories due to biological plausibility
 - ⇒ Easy understandable out of computer scientist community ¹
- ⇒ Interest for associative memories quantum implementation
 - ⇒ Also for its historical importance ²

1. C. Bauckhage, R. Sanchez, and R. Sifa. Problem solving with hopfield networks and adiabatic quantum computing. 2020.

2. M. Schuld, I. Sinayskiy, and F. Petruccione. The quest for a quantum neural network. Quantum Information Processing, 2014

3. C.F. Higham and A. Bedford. Quantum deep learning: Sampling neural nets with a quantum annealer. Nature Scientific Reports, 2021.

4. M. Sasdelli and T. Chin. Quantum annealing formulation for binary neural networks. Proceedings of the Digital Image Computing: Techniques and Applications, 2021.

Why dense associative networks in the annealer

- ⇒ No quantum memory capable of exponential storage on annealer
- ⇒ Interest in neuroscientific community for associative memories due to biological plausibility
 - ⇒ Easy understandable out of computer scientist community ¹
- ⇒ Interest for associative memories quantum implementation
 - ⇒ Also for its historical importance ²
- ⇒ Dense associative memories have a neural network interpretation
 - ⇒ Actually no proposal to implement neural network on QA apart from sampling ³ and binary shallow neural network ⁴
 - ⇒ No proposal for training deep neural networks on QA

1. C. Bauckhage, R. Sanchez, and R. Sifa. Problem solving with hopfield networks and adiabatic quantum computing. 2020.

2. M. Schuld, I. Sinayskiy, and F. Petruccione. The quest for a quantum neural network. Quantum Information Processing, 2014

3. C.F. Higham and A. Bedford. Quantum deep learning: Sampling neural nets with a quantum annealer. Nature Scientific Reports, 2021.

4. M. Sasdelli and T. Chin. Quantum annealing formulation for binary neural networks. Proceedings of the Digital Image Computing: Techniques and Applications, 2021.

Introductory concepts

Introductory concepts - dense associative memories

Symbols

$\mathbf{s}$ bipolar input vector

s_i single bipolar input

ξ^μ bipolar memory stored with index μ

ξ_i^μ single bipolar entry in memory with index μ

n length of bipolar inputs and memories

m number of stored memories

What is a dense associative memory¹?

$$E = - \sum_{\mu=1}^m F(\mathbf{s}^T \xi^\mu)$$

Markovian evolution of asynchronous dynamic

$$s_i^{(t+1)} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) - F \left(-\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) \right) \right]$$

¹ Hopfield J. Krotov D. Dense associative memory for pattern recognition. *Advances in Neural Information Processing Systems*, 2016.

² Löwe M. Uppgang S. Vermet F. Demircigil M., Heusel J. On a model of associative memory with huge storage capacity. *Journal of Statistical Physics*, 2017.

³ Lehner J. Seidl P. Widrich M. Adler T. Gruber L. Holzleitner M. Pavlovic M. Sandve K.G. Greiff V. Kreill D. Kopp M. Klaumbauer G. Brandstetter J. Hochreiter S. Ramsauer H., Schaf B. Hopfield network is all you need. *ArXiv Neural and Evolutionary Computing*, 2021.

Introductory concepts - dense associative memories

Symbols

$\mathbf{s}$ bipolar input vector

s_i single bipolar input

ξ^μ bipolar memory stored with index μ

ξ_i^μ single bipolar entry in memory with index μ

n length of bipolar inputs and memories

m number of stored memories

What is a dense associative memory¹?

$$E = - \sum_{\mu=1}^m F(\mathbf{s}^T \xi^\mu)$$

Markovian evolution of asynchronous dynamic

$$s_i^{(t+1)} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) - F \left(-\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) \right) \right]$$

⇒ dense associative memories have an exponential storage capacity²

⇒ dense associative memories could be used to implement attention in transformers³

¹ Hopfield J. Krotov D. Dense associative memory for pattern recognition. *Advances in Neural Information Processing Systems*, 2016.

² L'owe M. Upgang S. Vermet F. Demircigil M., Heusel J. On a model of associative memory with huge storage capacity. *Journal of Statistical Physics*, 2017.

³ Lehner J. Seidl P. Widrich M. Adler T. Gruber L. Holzleitner M. Pavlovic M. Sandve K.G. Greiff V. Kreill D. Kopp M. Klaumbauer G. Brandstetter J. Hochreiter S. Ramsauer H., Schafl B. Hopfield network is all you need. *ArXiv Neural and Evolutionary Computing*, 2021.

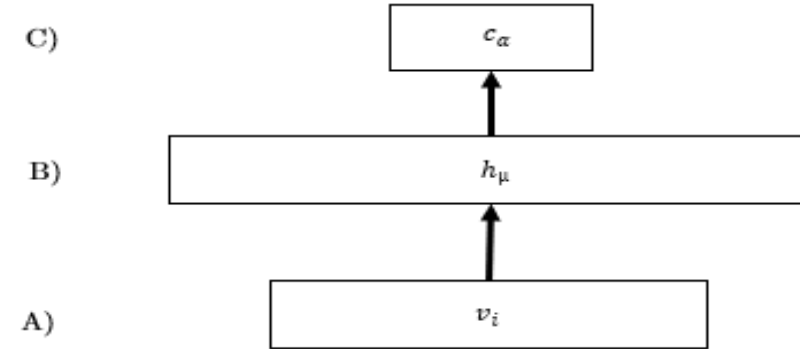
Introductory concepts - dense associative memories

Duality property ¹

Associative evolution



Feedforward evolution



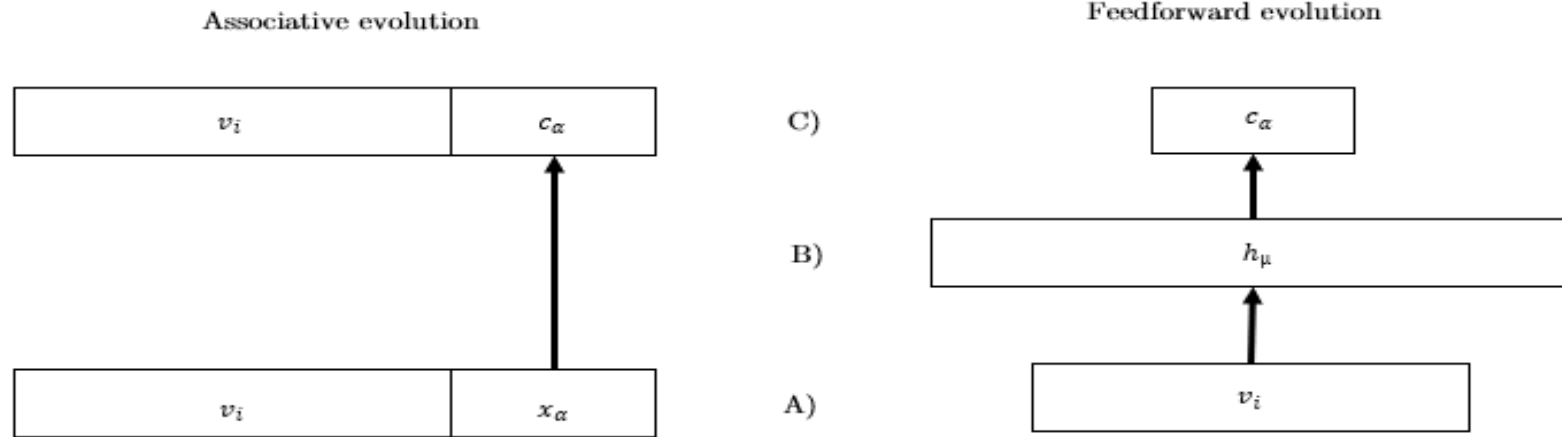
$$c_\alpha = \text{Sign} \left[\beta \sum_{\mu=1} \left(F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] = \text{Sign} \left[\sum_{\mu=1} \left(\xi_\alpha^\mu x_\alpha F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right]$$

1: Hopfield J. Krotov D. Dense associative memory for pattern recognition. *Advances in Neural Information Processing Systems*, 2016.

2. G. Iatropoulos, J. Brea, and W. Gerstner. Kernel memory networks: A unifying framework for memory modeling. 2022.

Introductory concepts - dense associative memories

Duality property ¹



$$c_\alpha = \text{Sign} \left[\beta \sum_{\mu=1} \left(F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] = \text{Sign} \left[\sum_{\mu=1} \left(\xi_\alpha^\mu x_\alpha F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right]$$

Relationships with kernel machines ²

$$\text{Sign} \left[\sum_{\mu=1} \left(\xi_\alpha^\mu x_\alpha F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] = \text{sgn} \left[\sum_{\mu=1}^m \alpha_i^\mu \xi_i^\mu k \left(\sum_{j \neq i}^n \xi_j^\mu s_j^{(t)} \right) - \theta_i \right] \quad \begin{aligned} k &= F' \\ \mathbf{A} &= \mathbf{1}, \\ \boldsymbol{\theta} &= \mathbf{0} \end{aligned}$$

1: Hopfield J. Krotov D. Dense associative memory for pattern recognition. *Advances in Neural Information Processing Systems*, 2016.

2. G. Iatropoulos, J. Brea, and W. Gerstner. Kernel memory networks: A unifying framework for memory modeling. 2022.

Introductory concepts – Quantum annealing

Quantum annealing

⇒ Metaheuristic to solve NP problem



Introductory concepts – Quantum annealing

Quantum annealing

⇒ Metaheuristic to solve NP problem

$$\mathcal{H}(t) = \mathcal{H}_F + \Gamma(t)\mathcal{H}_D$$

Final/problem Hamiltonian

$$= \sum_i h_i \sigma_i^z + \sum_{i < j} J_{ij} \sigma_i^z \sigma_j^z$$

Disordering/initial Hamiltonian

$$= \sum_i \sigma_i^x$$

⇒ An interesting relationship with Hopfield network

$$E = -\frac{1}{2} \sum_{i,j} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

d=1



d=2



Introductory concepts – Quantum annealing

Quantum annealing

⇒ Metaheuristic to solve NP problem

$$\mathcal{H}(t) = \mathcal{H}_F + \Gamma(t)\mathcal{H}_D$$

Final/problem Hamiltonian

$$= \sum_i h_i \sigma_i^z + \sum_{i < j} J_{ij} \sigma_i^z \sigma_j^z$$

transverse field coefficient

Disordering/initial Hamiltonian

$$= \sum_i \sigma_i^x$$

⇒ An interesting relationship with Hopfield network

$$E = -\frac{1}{2} \sum_{i,j} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

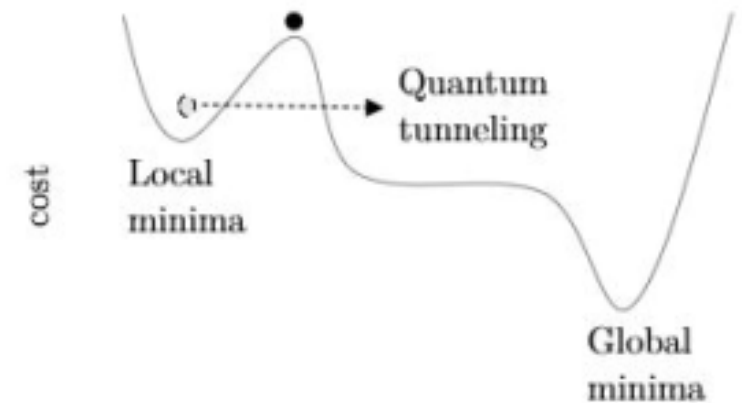
d=1

d=2

⇒ Quantum tunneling effect

⇒ Superposition¹

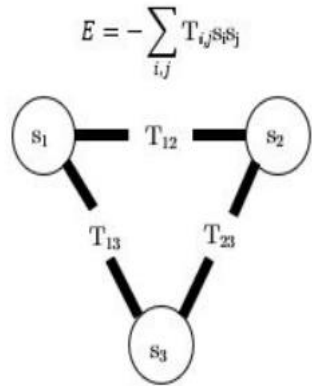
⇒ converge in polynomial time when the corresponding SA approach has an exponential lower bound²



First part – Quantum dense associative memory

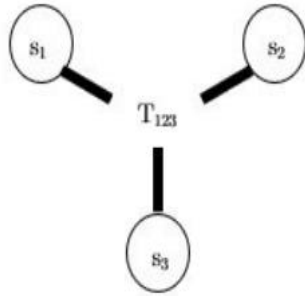
First part - Quantum dense associative memory

Some observations – handle more than binary interactions ²



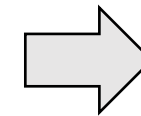
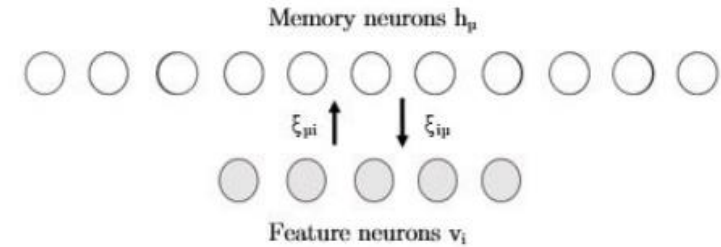
$$E = - \sum_{i,j} T_{ij} s_i s_j$$

$$E = - \sum_{\mu} F(\sum_i \xi_{\mu i} s_i) = - \sum_{i,j,k} T_{ijk} s_i s_j s_k$$



Representable
with

$$\begin{cases} \tau_f \frac{dv_i}{dt} = \sum_{\mu=1}^{n_h} \xi_{i\mu} f_{\mu} - v_i + i \\ \tau_h \frac{dh_{\mu}}{dt} = \sum_{i=1}^{n_f} \xi_{\mu i} g_i - h_{\mu} \end{cases}$$

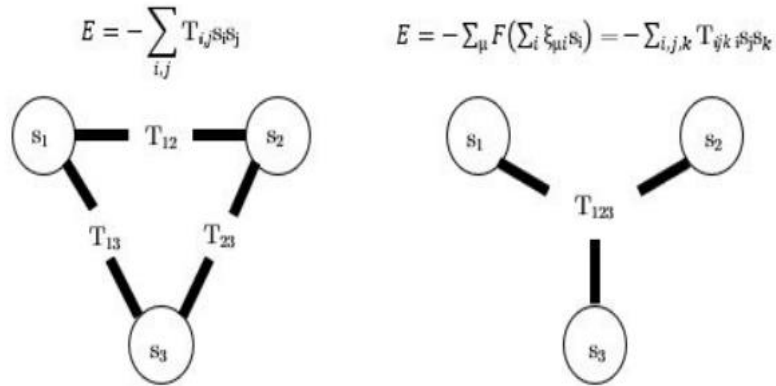


$$h_{\mu} = \sum_{i=1}^{n_f} \xi_{\mu i} g_i$$

Not possible with
bipolar / binary
variables

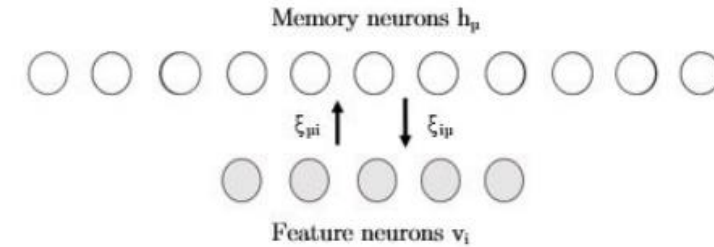
First part - Quantum dense associative memory

Some observations – handle more than binary interactions ²



Representable with

$$\begin{cases} \tau_f \frac{dv_i}{dt} = \sum_{\mu=1}^{n_h} \xi_{i\mu} f_{\mu} - v_i + i \\ \tau_h \frac{dh_{\mu}}{dt} = \sum_{i=1}^{n_f} \xi_{\mu i} g_i - h_{\mu} \end{cases}$$



$$h_{\mu} = \sum_{i=1}^{n_f} \xi_{\mu i} g_i$$

Not possible with bipolar / binary variables

Using advanced substitution techniques ¹ to decompose multiple interaction lead to an exponential complexity $m * \left(2^n - \binom{n}{2} - \binom{n}{1} - \binom{n}{0} \right)$

$$E = -\sum_{\mu=1}^m \exp(\mathbf{s}^T \boldsymbol{\xi}^{\mu}) = -\sum_{\mu=1}^m \prod_i^n e^{s_i \xi_i^{\mu}} = -\sum_{\mu=1}^m \prod_i^n \left(e^{-1} \frac{(1 - (\xi_i^{\mu} s_i))}{2} - e^{\frac{(-1 - (\xi_i^{\mu} s_i))}{2}} \right)$$

$\xi_i^{\mu k}$	s_i	$e^{\xi_i^{\mu k} s_i}$
+1	-1	e^{-1}
-1	+1	e^{-1}
+1	+1	e^1
-1	-1	e^1

First part - Quantum dense associative memory



Propose two cost effective strategies with different complexities

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \sum_{l=0}^{r-1} \gamma_{\mu,l} (t_{\mu,l} + \varepsilon_{\mu,r} \varepsilon_{\mu,l} - 2t'_{\mu,l} t_{\mu,l} - 2t'_{\mu,l} \varepsilon_{\mu,l} - 2t'_{\mu,l} \varepsilon_{\mu,r} + 4t'_{\mu,l}) - \sum_{l=0}^{r-1} 2^l t_{\mu,l} \left(\sum_{i=0}^{r-1} 2^i \varepsilon_{\mu,i} \right) \right) \Rightarrow m(3\log(n+1) + 1 + n)$$

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=1}^{n+1} (\varepsilon_{\mu,l}) (k_l)^2 \right) \Rightarrow m(2n + 1)$$

First part - Quantum dense associative memory

⇒ Propose two cost effective strategies with different complexities

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \sum_{l=0}^{r-1} \gamma_{\mu,l} (t_{\mu,l} + \varepsilon_{\mu,r} \varepsilon_{\mu,l} - 2t'_{\mu,l} t_{\mu,l} - 2t'_{\mu,l} \varepsilon_{\mu,l} - 2t'_{\mu,l} \varepsilon_{\mu,r} + 4t'_{\mu,l}) - \sum_{l=0}^{r-1} 2^l t_{\mu,l} \left(\sum_{i=0}^{r-1} 2^i \varepsilon_{\mu,i} \right) \right) \Rightarrow m(3\log(n+1) + 1 + n)$$

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=1}^{n+1} (\varepsilon_{\mu,l}) (k_l)^2 \right) \Rightarrow m(2n + 1)$$

⇒ Propose their extension to approximate all proposed type of nonlinearities

⇒ It is easy to obtain transformer energy

$$E = \frac{1}{2} \sum_{i=1}^{n_f} v_i^2 - \log \left(\sum_{\mu} \exp \left(\sum_i^n \xi_{\mu i} v_i \right) \right)$$

Second part – Quantum Feedforward Neural Networks

Second part – Quantum Feedforward Neural Networks

A cost effective way to train shallow neural network

⇒ Obtain an energy formulation for training via QA shallow neural networks invariant to number of data used

Comparing complexity of this algorithm

Willsh¹

$$D * n * d$$

Present work

$$2 * m * (|k| + n * d)$$

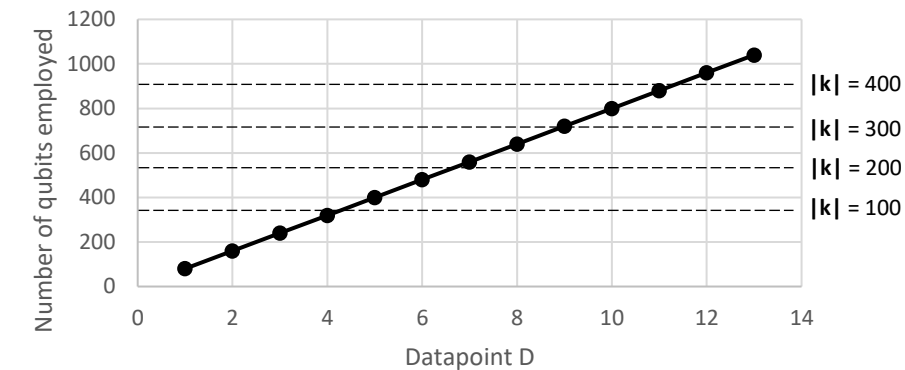
For the setting of¹

$$n = 40$$

$$d = 2$$

$$m = 1$$

Number of variables used by qSVM



1. D. Willsch, M. Willsch, H.D. Raedt, and K. Michielsen. Support vector machines on the d-wave quantum annealer. ArXiv Quantum Physics, 2016.

Second part – Quantum Feedforward Neural Networks

A cost effective way to train shallow neural network

⇒ Obtain an energy formulation for training via QA shallow neural networks invariant to number of data used

Comparing complexity of this algorithm

Willsh¹

$D * n * d$

Present work

$2 * m * (|k| + n * d)$

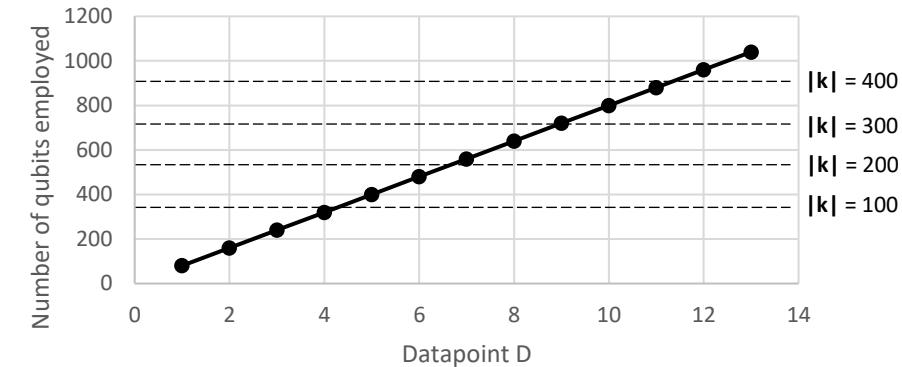
For the setting of¹

$n = 40$

$d = 2$

$m = 1$

Number of variables used by qSVM



How?

Duality
$$s_i^{(t+1)} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) - F \left(-\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) \right) \right] = \text{Sign} \left[\sum_{\mu=1}^m \left(\xi_i^\mu x_\alpha F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] = \text{Sign}[\hat{y}(\partial)]$$



Envelope in 0-1 loss

$$E = \sum_{\delta}^{|D|} (y_{\delta}^* - \hat{y}_{\delta})^2$$



$$E = \sum_{\delta}^D \left[\left(\beta \sum_{\mu=1}^m \left(\sum_{l=1}^{|k|} (\varepsilon'_{\mu,l}(k_l)^2 - \varepsilon_{\mu,l}(k_l)^2) \right) - y_{\delta}^* \right)^2 + \sum_{\mu=1}^m \gamma_{\mu}' \left(\sum_{l=1}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2 + \sum_{\mu=1}^m \gamma_{\mu}' \left(\sum_{l=1}^{|k|} \varepsilon'_{\mu,l} - 1 \right)^2 + \sum_{\mu=1}^m \gamma_{\mu} \left(- \sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^{\delta} - z_{\mu}' \right)^2 + \sum_{\mu=1}^m \gamma_{\mu} \left(\sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^{\delta} - z_{\mu} \right)^2 \right]$$

1. D. Willsch, M. Willsch, H.D. Raedt, and K. Michielsen. Support vector machines on the d-wave quantum annealer. ArXiv Quantum Physics, 2016.

Second part - Quantum Feedforward Neural Networks

⇒ Canonical loss minimization for deep neural network is impractical on the annealer due to multiple interactions explosion

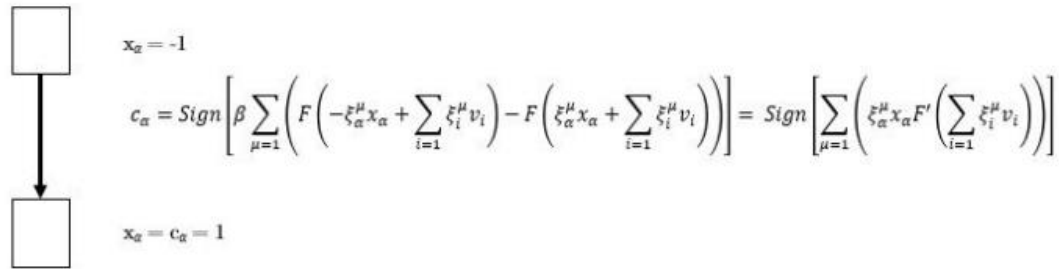
$$\mathcal{C} = \sum_{\delta}^{|D|} (y_{\delta}^* - c_{\alpha, \delta})^2 = \sum_{\delta}^{|D|} (y_{\delta}^* - f^n(f^{n-1}(\dots(f(v_{\delta}))))^2$$

Second part - Quantum Feedforward Neural Networks

- ⇒ Canonical loss minimization for deep neural network is impractical on the annealer due to multiple interactions explosion

$$C = \sum_{\delta} (y_{\delta}^* - c_{\alpha, \delta})^2 = \sum_{\delta} (y_{\delta}^* - f^n(f^{n-1}(\dots(f(v_{\delta}))))^2$$

- ⇒ Propose a cost effective way to embed deep neural network in annealer

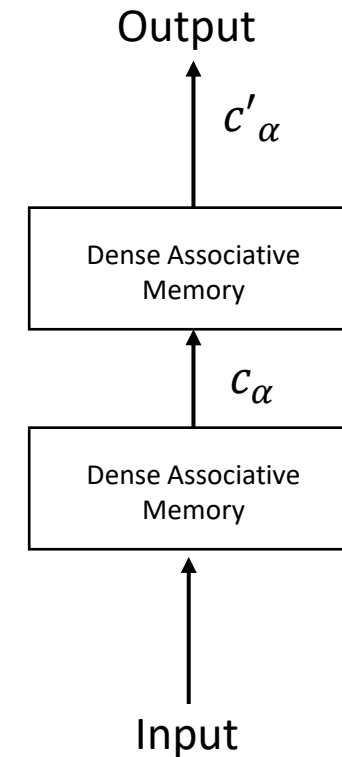


Number of variables for each layer:

$$L * m * (n_{\alpha} + (n + n_{\alpha}) * d + n * d^2 + (d + 1) * |k|)$$

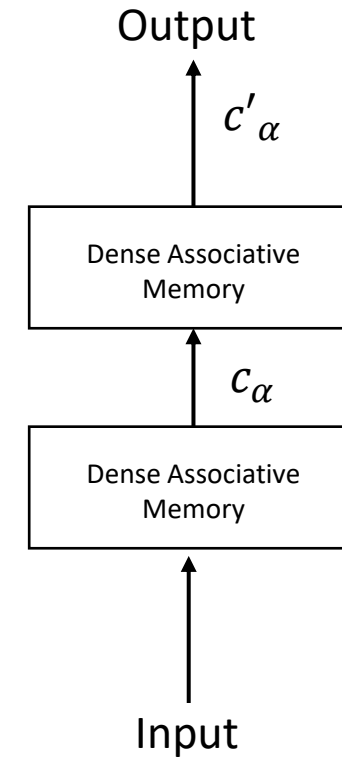
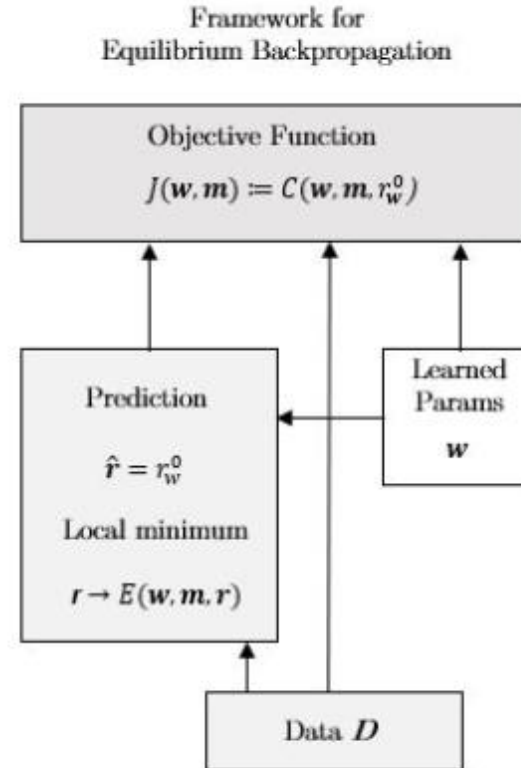
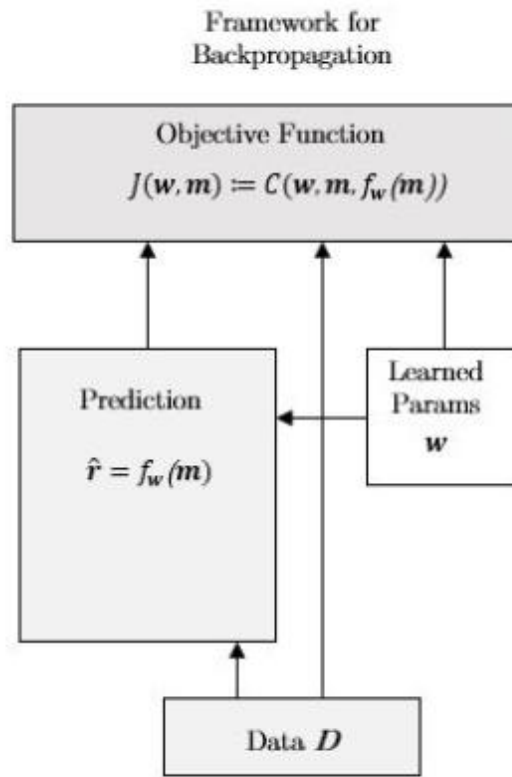
Example:

$L = 5, m = 2, n_{\alpha} = 6, n = 12, d = 3, k = 100$ lead to 5140 variable



Second part - Quantum Feedforward Neural Networks

⇒ Show how it is possible to train QA deep neural network via equilibrium propagation ¹



1. Bengio Y, Glorot X., Bordes A. Deep sparse rectifier neural networks. *Proceedings of Machine Learning Research*, 2011

Third part -Hybrid quantum approaches to neural networks

Third part-Hybrid quantum approach to neural networks

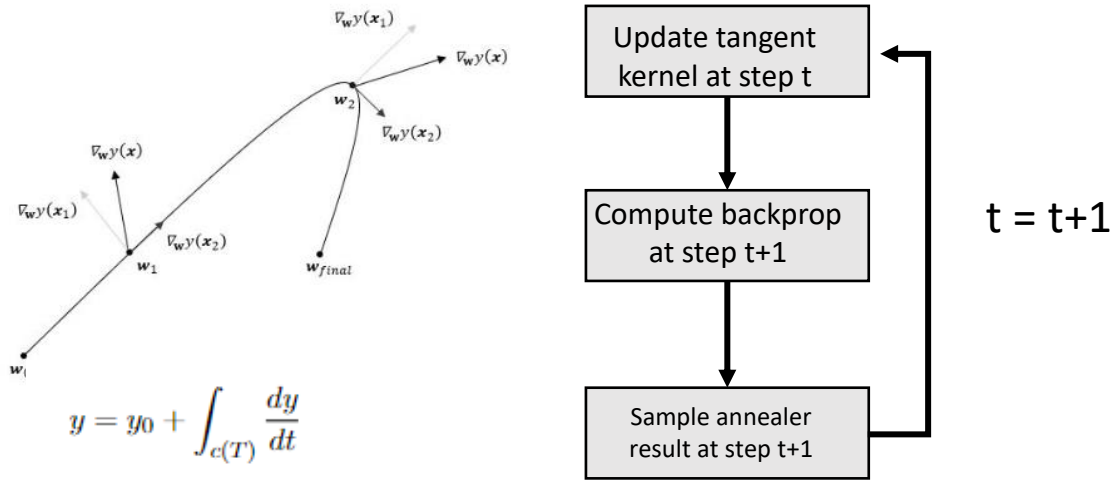
- ⇒ There is actually high interest for sampling neural networks via QA to speed up and evaluate input ¹
- ⇒ We presented a framework which allow to sample results layerwise and globally via dense associative memories
 - ⇒ Lead naturally to ensembling
 - ⇒ Propose how to implement skip connection inside this framework

1. C.F. Higham and A. Bedford. Quantum deep learning: Sampling neural nets with a quantum annealer. Nature Scientific Reports, 2021.

2. P. Domingos. Every model learned by gradient descent is approximately a kernel machine,. ArXiv Machine Learning, 2020.

Third part-Hybrid quantum approach to neural networks

- ⇒ There is actually high interest for sampling neural networks via QA to speed up and evaluate input ¹
- ⇒ We presented a framework which allow to sample results layerwise and globally via dense associative memories
 - ⇒ Lead naturally to ensembling
 - ⇒ Propose how to implement skip connection inside this framework
- ⇒ Show how one can train neural tangent kernel ² with annealer in hybrid classical/quantum fashion



1. C.F. Higham and A. Bedford. Quantum deep learning: Sampling neural nets with a quantum annealer. Nature Scientific Reports, 2021.

2. P. Domingos. Every model learned by gradient descent is approximately a kernel machine,. ArXiv Machine Learning, 2020.

Conclusions and further research

Conclusions

- ⇒ We provided cost effective dense associative memory energy formulation suitable for QA
- ⇒ We provided data invariant QA energy formulation for train shallow neural network
- ⇒ We provided a suitable QA deep neural network energy formulation and suggested way to train it via equilibrium propagation
- ⇒ We provided a way to sample deep neural networks via QA in a cost effective way
- ⇒ We provided a way to ensemble deep neural networks via QA
- ⇒ We provided a way to train deep neural network hybrid QA/classical fashion

Further research

- ⇒ Quantum attention layer
- ⇒ Apply the result to a real world problem (possibly multitarget)
- ⇒ Neural plausibility
- ⇒ Are there strategy to reduce ensemble correlations?

Thanks for the attention!

Extras

First part

Difficulties

$$E = - \sum_{\mu=1}^m \exp(s^T \xi^\mu) = - \sum_{\mu=1}^m \prod_i^n e^{s_i \xi_i^\mu} = - \sum_{\mu=1}^m \prod_i^n \left(e^{-1} \frac{(1 - (\xi_i^\mu s_i))}{2} - e^{\frac{(-1 - (\xi_i^\mu s_i))}{2}} \right)$$

$\xi_i^{\mu_k}$	s_i	$e^{\xi_i^{\mu_k} s_i}$
+1	-1	e^{-1}
-1	+1	e^{-1}
+1	+1	e^1
-1	-1	e^1

Noting that

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

Ishikawa substitution

$$y_1 = \sum_{i=1}^n x_i$$

$$y_2 = \sum_{i=1}^{n-1} \sum_{j=i+1}^n x_i x_j = \frac{y_1 (y_1 - 1)}{2}$$

if $a < 0$

$$ax_1 \cdots x_n = \min_{x_{n+1}} ax_{n+1} \{y_1 - (n - 1)\}$$

if $a > 0$

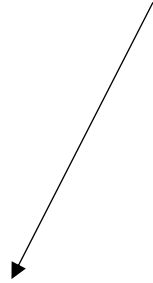
$$ax_1 \cdots x_n = a \min_{x_{n+1} \dots x_{n+n'}} \sum_{i=1}^{n'} x_{n+i} (a'_{i,n'} (-y_1 + 2i)) + ay_2$$

$$n' = \left\lceil \frac{n-1}{2} \right\rceil$$

$$a'_{i,n'} = \begin{cases} 1, & \text{if } d \text{ is odd and } i = n' \\ 2, & \text{otherwise} \end{cases}$$

First formulation

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \sum_{l=0}^{r-1} \gamma_{\mu,l} (t_{\mu,l} + \varepsilon_{\mu,r} \varepsilon_{\mu,l} - 2t'_{\mu,l} t_{\mu,l} - 2t'_{\mu,l} \varepsilon_{\mu,l} - 2t'_{\mu,l} \varepsilon_{\mu,r} + 4t'_{\mu,l}) - \sum_{l=0}^{r-1} 2^l t_{\mu,l} \left(\sum_{i=0}^{r-1} 2^i \varepsilon_{\mu,i} \right) \right)$$



Compute value of associative memory as 2-complement

$$y_{\mu} = \sum_{i=1}^n \xi_i^{\mu} * (2x_i - 1).$$

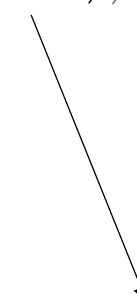
$$z_{\mu} = \sum_{l=0}^{r-1} 2^l \varepsilon_{\mu,l} - 2^r \varepsilon_{\mu,r}$$



Check its sign

$$\sum_{\mu=1}^m \sum_{l=0}^{r-1} \gamma_{\mu,l} \cdot (t_{\mu,l} - (1 - \varepsilon_{\mu,r}) \varepsilon_{\mu,l})^2$$

Compute the square



Second formulation

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=l^+}^{n+1} (\varepsilon_{\mu,l}) (k_l)^2 \right)$$

Compute value of associative memory given the dot product of two bipolar array admit only $n + 1$ values

$$z_{\mu} = \sum_{l=1}^{n+1} k_l \varepsilon_{\mu,l}$$

Force only single value is active

Output the square

Retrieve other nonlinearities

exponential

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=1}^{n+1} (\varepsilon_{\mu,l}) e^{k_l} \right)$$

tanh

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=1}^{n+1} (\varepsilon_{\mu,l}) \ln (1 + e^{k_l}) \right)$$

logistic

$$\cosh(x) = \frac{e^x + e^{-x}}{2}$$

$$\ln \left(\frac{e^x + e^{-x}}{2} \right) = \ln(1/2) + \ln(e^x + e^{-x}) = \ln(e^x (1 + e^{-2x})) = x + \ln(1 + e^{-2x})$$

Proceed as tanh

spherical

$$E(t) = - \sum_{\mu} F \left(\sum_i \xi_{\mu i} \frac{v_i}{\sqrt{\sum_j v_j^2}} \right)$$

Normalization is constant value

attention

$$E = \frac{1}{2} \sum_{i=1}^{n_f} v_i^2 - \log \left(\sum_{\mu} \exp \left(\sum_i \xi_{\mu i} v_i \right) \right)$$

1. Normalization is constant value
2. Logarithm is monotonic

Second part

Shallow neural network formulation

Duality $s_i^{(t+1)} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) - F \left(-\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) \right) \right] = \text{Sign} \left[\sum_{\mu=1}^m \left(\xi_\alpha^\mu x_\alpha F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] = \text{Sign}[\hat{y}(\partial)]$



Envelope in 0-1 loss

$$E = \sum_{\delta}^{|D|} (y_{\delta}^* - \hat{y}_{\delta})^2$$



$$E = \sum_{\delta}^D \left[\underbrace{\left(\beta \sum_{\mu=1}^m \left(\sum_{l=l^+}^{|k|} (\varepsilon'_{\mu,l}(k_l)^2 - \varepsilon_{\mu,l}(k_l)^2) \right) - y_{\delta}^* \right)^2}_{\text{Square of sum of variable of order 1}} + \sum_{\mu=1}^m \gamma'_{\mu} \left(\sum_{l=1}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2 + \sum_{\mu=1}^m \gamma'_{\mu} \left(\sum_{l=1}^{|k|} \varepsilon'_{\mu,l} - 1 \right)^2 + \sum_{\mu=1}^m \gamma_{\mu} \left(- \sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^{\delta} - z'_{\mu} \right)^2 + \sum_{\mu=1}^m \gamma_{\mu} \left(\sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^{\delta} - z_{\mu} \right)^2 \right]$$

Square of sum of
variable of order 1

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=l^+}^{n+1} (\varepsilon_{\mu,l} (k_l)^2) \right)$$

Difficulties (2)

Using shallow formulation in a three layer network

$$\begin{aligned}
 C = & \sum_{\delta} \left[\left(\beta' \sum_{\mu'=1}^D \sum_{l'=l'+1}^{|k'|} \left(\varepsilon'''_{\mu',l'} (k'_{l'})^2 - \varepsilon''_{\mu',l'} (k'_{l'})^2 \right) - y_{\delta}^* \right)^2 \right. \\
 & + \sum_{\mu'=1} \gamma''_{\mu'} \left(\sum_{l'=1}^{|k|} \varepsilon'''_{\mu,l'} - 1 \right)^2 + \sum_{\mu'=1} \gamma''_{\mu'} \left(\sum_{l'=1}^{|k'|} \varepsilon''_{\mu,l'} - 1 \right)^2 \\
 & + \sum_{\mu=1} \gamma'_{\mu} \left(\sum_{l=1}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2 + \sum_{\mu=1} \gamma'_{\mu} \left(\sum_{l=1}^{|k|} \varepsilon'_{\mu,l} - 1 \right)^2 \\
 & + \sum_{\mu'=1}^{m'} \gamma'''_{\mu} \left(- \sum_{l'=0}^{d'-1} b^{l'} s'_{0,l'} + \sum_{i'=1}^{n'} \sum_{l'=0}^{d'-1} b^{l'} s'_{i',l'} \beta \sum_{\mu=1}^{|k|} \sum_{l=l'+1}^{|k|} \left(\varepsilon'_{\mu,l} (k_l)^2 - \varepsilon_{\mu,l} (k_l)^2 \right) - z'''_{\mu'} \right)^2 \\
 & + \sum_{\mu'=1}^{m'} \gamma'''_{\mu} \left(- \sum_{l'=0}^{d'-1} b^{l'} s'_{0,l'} + \sum_{i'=1}^{n'} \sum_{l'=0}^{d'-1} b^{l'} s'_{i',l'} \beta \sum_{\mu=1}^{|k|} \sum_{l=l'+1}^{|k|} \left(\varepsilon'_{\mu,l} (k_l)^2 - \varepsilon_{\mu,l} (k_l)^2 \right) - z''_{\mu'} \right)^2 \\
 & \left. + \sum_{\mu=1}^m \gamma_{\mu} \left(- \sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^{\delta} - z'_{\mu} \right)^2 + \sum_{\mu=1}^m \gamma_{\mu} \left(\sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^{\delta} - z_{\mu} \right)^2 \right]
 \end{aligned}$$

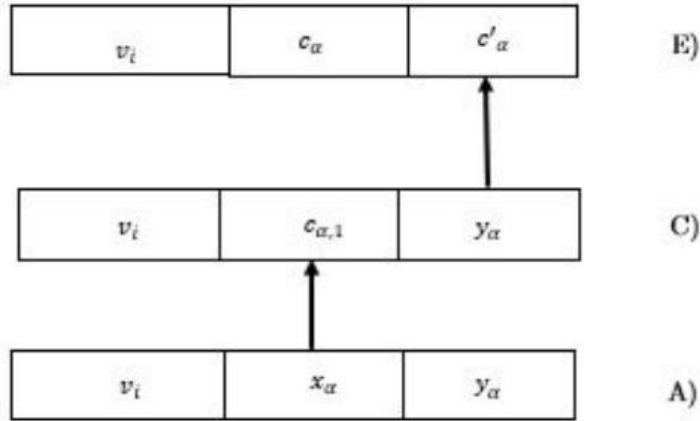
Envelop first layer result in second layer result require guarantee that output of second layer depends on first layer

Notice second order interactions into a square (which become fourth order interaction)

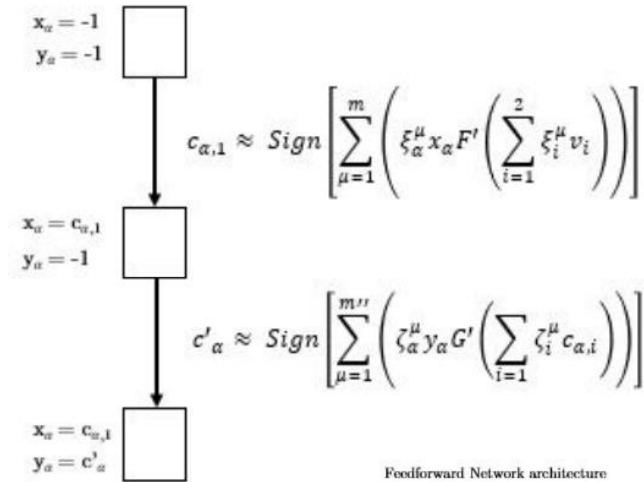
Deep neural network formulation (1)

An example of a three layer network

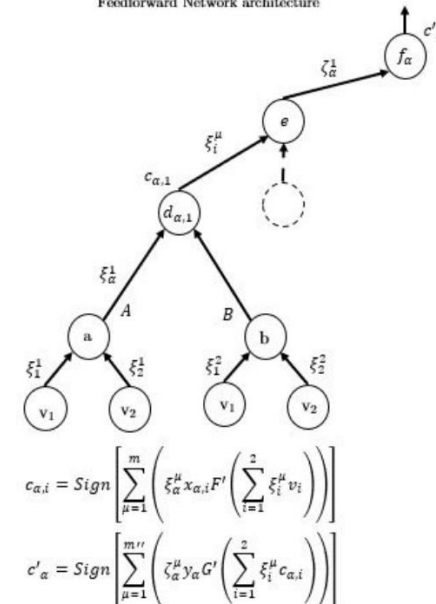
Associative evolution



Markov dynamic of classic synchronizable network



Feedforward Network architecture



$$E_{TOT} = E(E_{1,1}, W) + E(E_{1,2}, W) + E(E_{2,1}, W)$$

Deep neural network formulation (2)

An example of a three layer network

$$E_{TOT} = E(E_{1,1}, W) + E(E_{1,2}, W) + E(E_{2,1}, W)$$

Substituting $E(E_{1,1}, W)$ and $E(E_{1,2}, W)$

$$\begin{aligned} E_{TOT} = & \sum_{\mu=1}^m \gamma_{\mu} \left(\sum_{l=0}^{d-1} b^l s_{0,l} c_{\alpha,1} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_{\mu} \right)^2 - \beta \sum_{\mu=1}^m \sum_{l=1}^{|k|} \varepsilon_{\mu,l} (k_l)^2 \\ & + \sum_{\mu'=1}^{m'} \gamma'_{\mu'} \left(\sum_{l'=0}^{d'-1} b^{l'} s'_{0,l'} c_{\alpha,2} + \sum_{i=1}^n \sum_{l'=0}^{d'-1} b^{l'} s'_{i,l'} v_i - z_{\mu'} \right)^2 - \beta' \sum_{\mu'=1}^{m'} \sum_{l'=1}^{|k'|} \varepsilon'_{\mu',l'} (k'_{l'})^2 \\ & + E(E_{2,1}, W) \end{aligned}$$

Last component

$$E(E_2, W) = \sum_{\mu=1}^{m''} \gamma''_{\mu} \left(\sum_{l=0}^{d''-1} b^l s''_{0,l} c'_{\alpha} + \sum_{i=1}^2 \sum_{l=0}^{d''-1} b^l s''_{i,l} c_{\alpha,i} - z''_{\mu} \right)^2 - \beta'' \sum_{\mu=1}^{m''} \sum_{l=1}^{|k''|} \varepsilon''_{\mu,l} (k''_l)^2$$

Deep neural network formulation (3)

$$E(E_1, W) = \sum_{\mu=1}^m \gamma_{\mu} \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_{\alpha} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_{\mu} \right)^2 + \sum_{\mu=1}^m \gamma'_{\mu} \left(\sum_{l=1}^{|\mathbf{k}|} \varepsilon_{\mu,l} - 1 \right)^2 - \beta \sum_{\mu=1}^m \sum_{l=l+}^{|\mathbf{k}|} \varepsilon_{\mu,l} (k_l)^2$$

↓ Develop square

$$\begin{aligned} \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_{\alpha} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_{\mu} \right)^2 &= \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_{\alpha} \right)^2 + \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right)^2 \\ &\quad + z_{\mu}^2 + 2 \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_{\alpha} \right) \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right) \\ &\quad - 2 \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_{\alpha} \right) (z_{\mu}) - 2 \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right) z_{\mu} \end{aligned}$$

↓ Only fourth and fifth terms have more than binary interaction

$$\begin{aligned} = & \left[\left(\sum_{l=0}^{d-1} b^{2l} (s_{0,l} s_{\alpha})^2 + 2 \sum_{i < j}^{d-1} b^{i+j} s_{0,i} s_{0,j} \right) + \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right)^2 + z_{\mu}^2 \right. \\ & + \left(2 \sum_{i=1}^n \sum_{l=0}^{d-1} \sum_{l'=0}^{d-1} v_i b^{l'+l} s_{0,l'} s_{\alpha} s_{i,l} \right) - \left(2 \sum_{l=0}^{d-1} \sum_{l'=1}^{|\mathbf{k}|} b^l s_{0,l} s_{\alpha} \varepsilon_{\mu,l'} k_{l'} \right) \\ & \left. - 2 \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right) z_{\mu} \right] \end{aligned}$$

$\nearrow$
 $n * d^2$

$\longrightarrow$
 $d * |\mathbf{k}|$

Auxilliary
variables
introduced

Deep neural network formulation (4)

$$L * m * (n_{\alpha} + (n + n_{\alpha}) * d + n * d^2 + (d + 1) * |k|)$$

Where:

L is number of layer

n_{α} is number of classification output of each layer

n is number of input to each layer

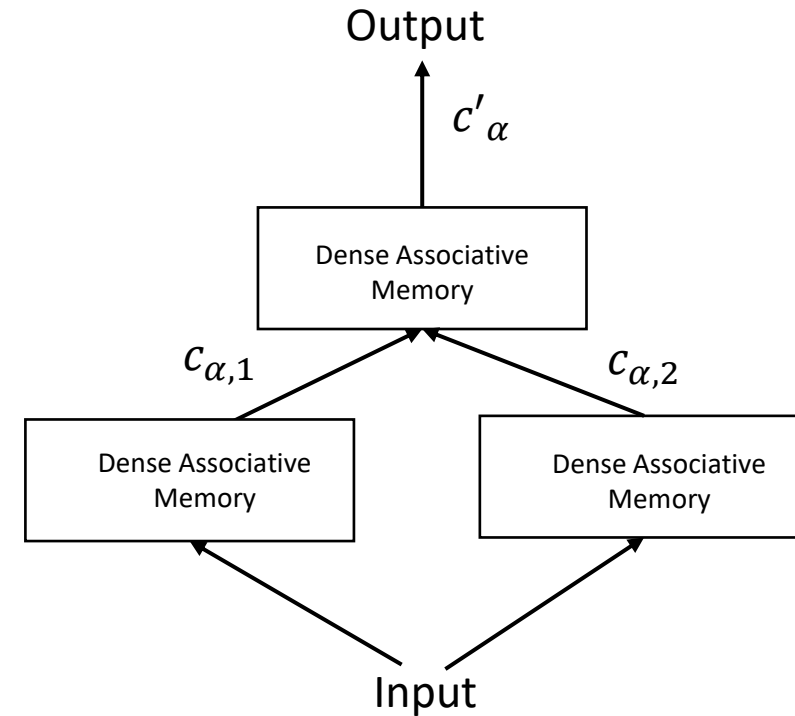
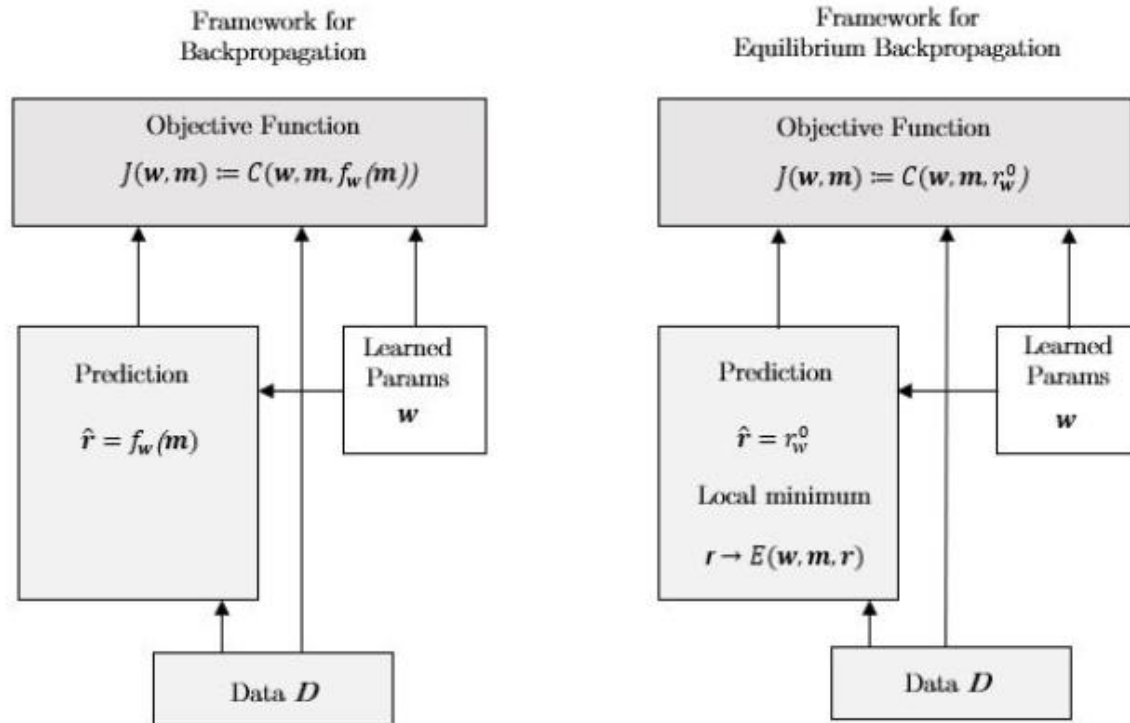
d is number of variables used to model weights

m is number of memories for each dense associative memory

Assumptions

Each layer has same number of input and output neurons

Equilibrium propagation applied to qDNN (1)



$$E_{TOT} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_{\alpha}^{\mu} x_{\alpha,2} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] + \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_{\alpha}^{\mu} x_{\alpha,1} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] \\ + \text{Sign} \left[\sum_{\mu=1}^{m'} \left(F \left(\xi_{\alpha}^{\mu'} x'_{\alpha} + \sum_{i=1}^{n'} \xi_i^{\mu'} c_{\alpha,i} \right) \right) \right]$$

Equilibrium propagation applied to qDNN (2)

$$E_{TOT} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_{\alpha}^{\mu} x_{\alpha,2} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] + \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_{\alpha}^{\mu} x_{\alpha,1} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] \\ + \text{Sign} \left[\sum_{\mu=1}^{m'} \left(F \left(\xi_{\alpha}^{\mu'} x'_{\alpha} + \sum_{i=1}^{n'} \xi_i^{\mu'} c_{\alpha,i} \right) \right) \right]$$

Compute derivatives in respect of weights of associative memories

$$\frac{\partial E_{TOT}}{\partial \mathbf{w}} = \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \mathbf{w}^{1,1}} + \frac{\partial E(E_{1,2}, \mathbf{w}^{1,2})}{\partial \mathbf{w}^{1,2}} + \frac{\partial E(E_2, \mathbf{w}^2)}{\partial \mathbf{w}^2}$$

For a generic associative memory

$$\frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \mathbf{w}^{1,1}} = \left[\frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_1^{\mu}}, \dots, \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_N^{\mu}}, \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_{\alpha,1}^{\mu}} \right]$$

Finally the derivatives

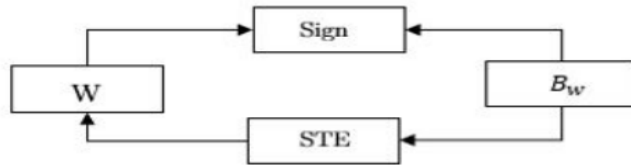
$$\frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_i^{\mu}} = \sum_{\mu=1}^m \left(2 \left(\xi_{\alpha,1}^{\mu} x_{\alpha,1} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) v_i \quad \text{For input layer}$$

$$\frac{\partial E(E_1, \mathbf{w}^2)}{\partial \xi_i^{\mu''}} = \sum_{\mu=1}^{m''} \left(2 \left(\xi_{\alpha}^{\mu''} x'_{\alpha} + \sum_{i=1}^{n'} \xi_i^{\mu''} c_{\alpha,i} \right) \right) c_{\alpha,i} \quad \text{For output layer}$$

Equilibrium propagation applied to qDNN (2)

How compute derivative of $\text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_{\alpha}^{\mu} x_{\alpha,2} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] = \text{Sign} \left[\sum_{\mu=1}^m \left(\text{Rep 2} \left(\xi_{\alpha}^{\mu} x_{\alpha,2} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right]$

Straight through estimation

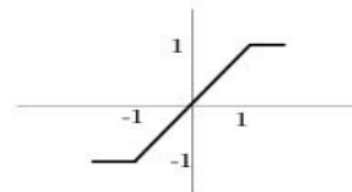


Sign: Forward



$$\text{Sign}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

Approximate Sign: Backward

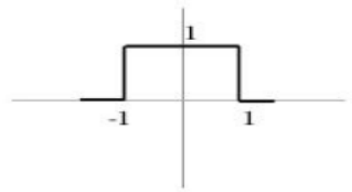


$$\text{Approx}(x) = \begin{cases} x & \text{if } x \geq -1 \text{ and } x \leq 1 \\ -1 & \text{if } x < -1 \\ 1 & \text{otherwise} \end{cases}$$

Zero derivative issue: Backward



Derivative Approximate Sign: Backward



$$\text{STE}(x) = \frac{\delta \text{Approx}(x)}{\delta x} = \begin{cases} 1 & \text{if } x \geq -1 \text{ and } x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Subgradient optimization

Derivative of nonlinearity Rep2 $\begin{cases} 2x & \text{if } x \geq 0 \\ 0 & \text{if } x \leq 0 \end{cases}$

Using subgradient optimization you can model as $2x$

Third part

Ensembling

A shallow network can be approximated as

$$c_\alpha = \text{Sign} \left[\beta \sum_{\mu=1}^m \left(F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(+\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right]$$

$$\approx \text{sign} \left[\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right]$$

Notice that only this component approximate forward

$$\beta \sum_{\mu=1}^m F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) - \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right)$$

The second layer can be retrieved as

$$c'_\alpha = \text{sign} \left(F \left(-x'_\alpha \xi_\alpha^{\mu'} + \left(F \left(-x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \right. \\ \left. - F \left(x'_\alpha \xi_\alpha^{\mu'} + \left(F \left(-x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \right)$$

Insert inside

Ensembling(2)

$$c'_\alpha = \text{sign} \left(F \left(-x'_\alpha \xi_\alpha^{\mu'} + \left(F \left(-x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) - F \left(x'_\alpha \xi_\alpha^{\mu'} + \left(F \left(-x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \right)$$

What is the number of variable employed if implemented in annealer?

All depends in how you treat F nonlinearity: consider a square approximation of a ReLU ¹

$$F = \frac{1}{3} \left(\sum_{i=1}^n \xi_i^\mu v_i \right)^3 + \left(\sum_{i=1}^n \xi_i^\mu v_i \right)^2$$

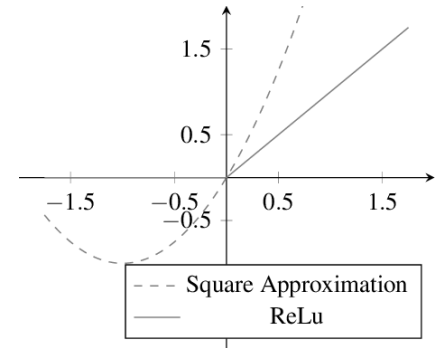
First layer

$$\begin{aligned} \beta \sum_{\mu=1}^m F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) - \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) &\longrightarrow \sum_{\mu=1}^m \frac{1}{3} \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right)^3 + \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right)^2 - \frac{1}{3} \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right)^3 + \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right)^2 \\ &\longrightarrow \sum_{\mu=1}^m a_\mu x_\alpha \end{aligned}$$

Second layer

$$\sum_{\mu'=1}^{m'} \frac{1}{3} \left(-\xi_\alpha^{\mu'} x'_\alpha + \sum_{i=1}^n \xi_i^{\mu'} \sum_{\mu=1}^m a_\mu x_\alpha \right)^3 + \left(-\xi_\alpha^{\mu'} x'_\alpha + \sum_{i=1}^n \xi_i^{\mu'} \sum_{\mu=1}^m a_\mu x_\alpha \right)^2 - \frac{1}{3} \left(\xi_\alpha^{\mu'} x'_\alpha + \sum_{i=1}^n \xi_i^{\mu'} \sum_{\mu=1}^m a_\mu x_\alpha \right)^3 + \left(\xi_\alpha^{\mu'} x'_\alpha + \sum_{i=1}^n \xi_i^{\mu'} \sum_{\mu=1}^m a_\mu x_\alpha \right)^2$$

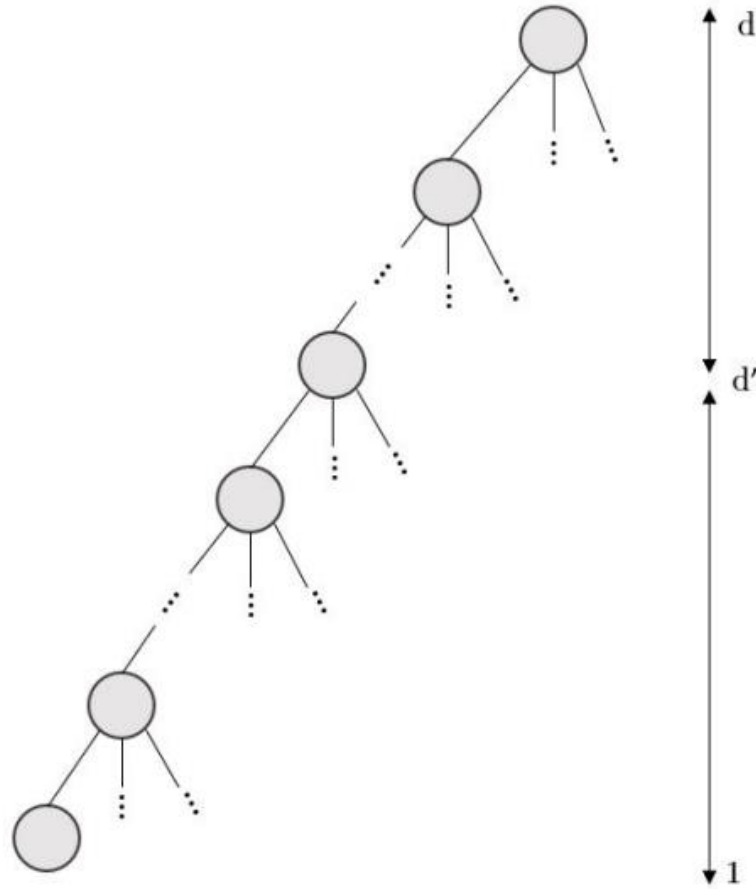
Only binary interactions involved



¹ Security for Distributed Deep Neural Networks Towards Data Confidentiality & Intellectual Property Protection, Gomez, L., Wilhelm, M., Marquez, J., Duverger, P., 2019

Ensembling(3)

You can repeat this process



Using the approach presented before

$$c_d = \text{Sign} \left[\sum_{\mu} \beta \left(F \left(-\xi_d^{\mu} x_d + \sum_i^{n_d} \xi_i^{\mu} E_i \right) - F \left(+\xi_d^{\mu} x_d + \sum_i^{n_d} \xi_i^{\mu} E_i \right) \right) \right]$$

$$c_{1,d'} = \text{Sign} \left[\sum_{\mu} \beta \left(F \left(-\xi_{1,d'}^{\mu} x_{1,d'} + \sum_i^{n_{d'}} \xi_i^{\mu} v_d \right) - F \left(+\xi_{1,d'}^{\mu} x_{1,d'} + \sum_i^{n_{d'}} \xi_i^{\mu} v_{d'} \right) \right) \right] = \text{Sign} [E_{1,d'}]$$

Using neural tangent kernel

$$K_2(\mathbf{x}^*, \mathbf{x}_i^*) = \int_{c(t)} \sum_{j=d'}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t} dt$$

$$K_1(\mathbf{x}^*, \mathbf{x}_i^*) = \int_{c(t)} \sum_{j=1}^{d'} \frac{\partial y_{d'}}{\partial w_j} \frac{\partial w_j}{\partial t} dt$$

Skip connection

Energy update is equal to

In a bipolar variable setting this is equal to

$$F\left(-\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i\right) - F\left(\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i\right) \longrightarrow F\left(-\xi_{\alpha}^{\mu} x_{\alpha} + \sum_{i=1}^n \xi_i^{\mu} v_i\right) - F\left(\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i\right)$$

Notice that if x_{α} not change sign meaning $x_{\alpha} = -1$ then

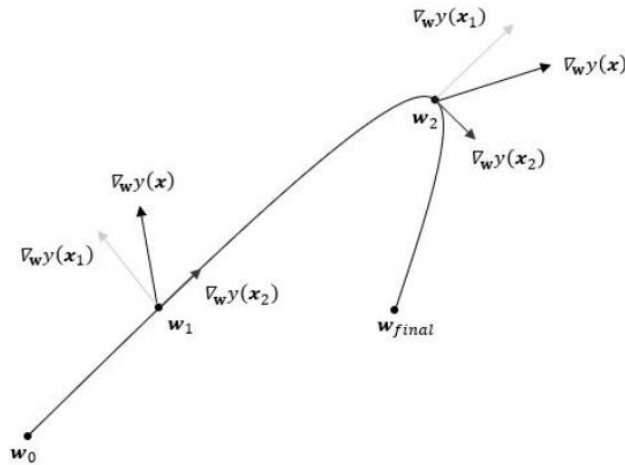
$$F\left(-\xi_{\alpha}^{\mu} x_{\alpha} + \sum_{i=1}^n \xi_i^{\mu} v_i\right) - F\left(\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i\right) = F\left(-\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i\right) - F\left(-\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i\right) = 0$$

To implement skip connection formulate energy as

$$E = F\left(-x'_{\alpha} \xi_{\alpha}^{\mu} + \left(F\left(x_{\alpha} \xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i\right) - C\right) + \sum_{i=1}^n \xi_i^{\mu} v_i\right) - C$$
$$C = F\left(-\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i\right)$$

Hybrid training (1)

Neural tangent kernel



A model output can be seen as

$$y = y_0 + \int_{c(T)} \frac{dy}{dt}$$

If the model is differentiable then

$$\frac{dy}{dt} = \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{dw_j}{dt}$$

given a model trained with gradient descent, it holds

$$\frac{w^{s+1} - w^s}{\epsilon} = -\nabla_w L(w^s),$$

Noticing that

$$\begin{aligned} \frac{dy}{dt} &= \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t} \\ &= \sum_{j=1}^d \frac{\partial y}{\partial w_j} \left(-\frac{\partial L}{\partial w_j} \right) \end{aligned}$$

Then

$$\begin{aligned} \frac{dy}{dt} &= \sum_{j=1}^d \frac{\partial y}{\partial w_j} \left(-\sum_{i=1}^m \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial w_j} \right) \\ &= -\sum_{i=1}^m \frac{\partial L}{\partial y_i} \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial y_i}{\partial w_j} \end{aligned}$$

Hybrid training (2)

$$\begin{aligned}\frac{dy}{dt} &= \sum_{j=1}^d \frac{\partial y}{\partial w_j} \left(- \sum_{i=1}^m \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial w_j} \right) \\ &= - \sum_{i=1}^m \frac{\partial L}{\partial y_i} \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial y_i}{\partial w_j}\end{aligned}$$

defining

$$L' (y_i^*, y_i) = \partial L / \partial y_i$$

$$K_{f,v}^g (\mathbf{x}, \mathbf{x}') = \nabla_{\mathbf{w}} f_{\mathbf{w}} (\mathbf{x}) \nabla_{\mathbf{w}} f_{\mathbf{w}} (\mathbf{x}'),$$

Returning to original formula

$$\begin{aligned}\lim_{\epsilon \rightarrow 0} y &= y_0 - \int_{c(T)} \sum_{i=1}^m L' (y_i^*, y_i) K_{f,w(t)}^g (\mathbf{x}, \mathbf{x}_i) dt, = y_0 - \sum_{i=1}^m \left(\frac{\int_{c(t)} K_{f,w(t)}^g (\mathbf{x}, \mathbf{x}_i) L' (y_i^*, y_i) dt}{\int_{c(t)} K_{f,w(t)}^g (\mathbf{x}, \mathbf{x}_i) dt} \right) \int_{c(t)} K_{f,w(t)}^g (\mathbf{x}, \mathbf{x}_i) dt \\ &= y_0 - \sum_{i=1}^m \bar{L}' (y_i^*, y_i) K_{f,c}^p (\mathbf{x}, \mathbf{x}_i)\end{aligned}$$

Which is a kernel machine

$$\lim_{\epsilon \rightarrow 0} y = \sum_{i=1}^m a_i K (\mathbf{x}, \mathbf{x}_i) + b$$

where

$$K (\mathbf{x}, \mathbf{x}_i) = K_{f,c}^p (\mathbf{x}, \mathbf{x}_i)$$

$$, a_i = -\bar{L}' (y_i^*, y_i),$$

Hybrid training (3)

Which is a kernel machine

$$\lim_{\epsilon \rightarrow 0} y = \sum_{i=1}^m a_i K(\mathbf{x}, \mathbf{x}_i) + b$$

where

$$K(\mathbf{x}, \mathbf{x}_i) = K_{f,c}^p(\mathbf{x}, \mathbf{x}_i)$$

$$a_i = -\bar{L}'(y_i^*, y_i),$$

And can be embedded in a Dense associative memory as

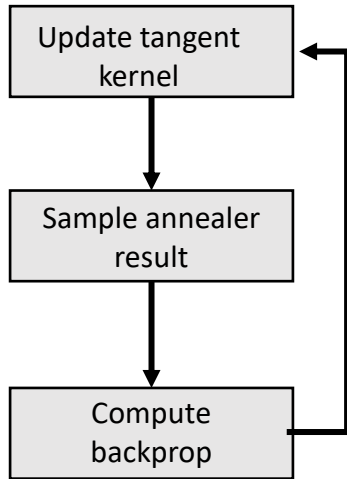
$$F = \int K_{f,c}^p(\mathbf{x}^*, \mathbf{x}_i^*)$$

$$\mathbf{x}^* = [x_1, \dots, x_d, x_\alpha]$$

$$\mathbf{x}_i^* = [x_1^i, \dots, x_d^i, a_i]$$

$$c_\alpha \approx \text{Sign} \left[\sum_{i=1}^m a_i K(\mathbf{x}, \mathbf{x}_i) + b \right]$$

Hybrid training (4)



How treat neural tangent kernel if a weight is optimized in a discrete fashion

$$K(\mathbf{x}^*, \mathbf{x}_i^*) = \int_{c(t)} \sum_{j=1}^d \frac{\partial y}{\partial w_j(t)} \frac{\partial w_j(t)}{\partial t} dt$$

$$= \sum_{j=1}^d \int_{c(t)} \frac{\partial y}{\partial w_j(t)} \frac{\partial w_j(t)}{\partial t} dt$$

Apply the line integral

$$\int_{c(t)} f(w_1(t), \dots, w_d(t)) dw_j = \int_{t_1}^{t_2} f(w_1(t), \dots, w_d(t)) w_j'(t) dt$$

$$= \sum_{j=1}^d \int_{c(t)} \frac{\partial y}{\partial w_j(t)} (w_j^{t+1} - w_j^t) dt$$

Split output in two phase

$$y'_0 = y_0 - \int_{C_1(t')} \sum_{i=1}^m L'(y_i^*, y_i) K_{f, w(t')}^g(\mathbf{x}, \mathbf{x}_i) dt' + \int_{C_2(t'')} \sum_{j=1}^d \frac{\partial y}{\partial w_j} (w_j^{t'''} - w_j^{t''})$$

Where

$$T = T' + T''$$

$$T' = [0, t'), T'' = [t', t'')$$

Get finally the following energy

$$E = \sum_{k=1}^m \left(y_k^* - \hat{y}(\mathbf{x}_k) \right)^2 + r$$

$$\hat{y}(\mathbf{x}_k) = y_{t-1} + \sigma_1 \int_{C_1(t)} \sum_{i=1}^m L'(y_i^*, y_i) K_{f, w(t)}^g(\mathbf{x}_k, \mathbf{x}_i) dt + \sigma_2 \int_{C_2(t)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} (w_j^{t+1} - w_j^t) dt$$

Advantages

- iterative
- Incremental
- parallelizable
- Can only improve results escaping local minima
- Every step generate a new network which can be used for ensemble